

Safe-Escape MPPI: Coordinating Online Local-Minima Escape with Control-Barrier-Function Safety in a Nav2-Native Controller

Jung Mo Kang

Abstract—The Model Predictive Path Integral (MPPI) controller shipped with ROS 2 Nav2 is among the strongest deployable local controllers for mobile robots, yet in narrow, crowded, dynamic spaces it is prone to local-minima entrapment and enforces collision avoidance only through soft costs, with no formal safety guarantee. Prior work treats these weaknesses in isolation, and none ships as a deployable Nav2 controller. We present Safe-Escape MPPI (SE-MPPI), a single Nav2-native controller that unifies online local-minima detection-and-escape with a dynamic-obstacle control-barrier-function (CBF) safety filter, reconciled by a coordinator that modulates the barrier’s class- $\mathcal{K}$ gain on detected entrapment. Forward invariance is preserved for any positive gain, so an escape maneuver admitted by a raised gain stays certified-safe. In a randomized 1,200-trial 2D benchmark mirroring the controller math one-to-one, the detection-and-escape layer is the decisive contribution on both trap families, with the free-space gap search load-bearing. The escape-safety coordination instead matches an independent escape+CBF stack – a null result: outcomes are insensitive to the barrier-gain schedule even where the barrier engages – so we claim it as a certified-safe mechanism, not an empirical gain. The controller also runs inside a live ROS 2 Jazzy + Nav2 + Gazebo stack, and we report the deployment faults live integration exposes.

Index Terms—Model predictive control, control barrier functions, collision avoidance, local-minima escape, motion and path planning, autonomous navigation, ROS 2 Nav2.

I. Introduction

Autonomous mobile robots increasingly operate in cluttered, dynamic indoor environments – warehouses, hospitals, service floors – where the local controller must be simultaneously fast, non-conservative, and safe. Within the ROS 2 Nav2 ecosystem, the MPPI controller (`nav2_mppi_controller`) [1] has become the de-facto high-performance choice: it supports differential, omnidirectional, and Ackermann robots and runs at 50+ Hz on CPU by sampling `batch_size` rollouts, scoring them with a bank of critics, and combining them with an information-theoretic softmax [2].

Two weaknesses, however, recur exactly in the environments that matter most:

- 1) Local-minima entrapment. A finite prediction horizon (by default $56 \times 0.05\text{s} \approx 2.8\text{s}$) and zero-mean Gaussian sampling cause the robot to stall

or oscillate in non-convex traps. Nav2’s only mitigations are always-on cost-shaping heuristics (`PreferForwardCritic`, `TwirlingCritic`) that neither detect entrapment nor guarantee escape.

- 2) No formal safety. Avoidance is encoded as soft costs (`ObstaclesCritic`, `CostCritic`); there is no forward-invariance guarantee, and crucially no prediction of dynamic-obstacle motion – the costmap is a present-time snapshot and `collision_monitor` is purely reactive.

These weaknesses interact: being conservative for safety invites entrapment, and escaping aggressively invites collision. We argue they should be solved together, in one deployable controller. Fig. 2(a) (Sec. VI-A) illustrates the first weakness and SE-MPPI’s response: stock MPPI stalls before a U-shaped trap while SE-MPPI detects the entrapment and rounds it under the certified-safe coordination of Sec. IV-E.

Contributions.

- C1 (system). To our knowledge, the first Nav2-native local controller plugin that combines a CBF safety filter with online local-minima escape; packaged as a `nav2_core::Controller` plus a pluginlib MPPI critic.
- C2 (algorithm). An escape-safety coordination mechanism that modulates the CBF class- $\mathcal{K}$ gain on detected entrapment, letting the robot escape without losing forward invariance (proven in Sec. IV-E), with a TTC override that re-prioritizes safety when a dynamic obstacle’s time-to-collision is imminent. We claim C2 as a certified-safe mechanism: in our 2D benchmark it is empirically indistinguishable from an independent escape+CBF stack (Sec. VI-C1) – outcomes proved insensitive to the barrier-gain schedule even where the barrier engages.
- C3 (dynamic safety). Integration of a look-ahead-point CBF safety filter for a differential-drive robot, fed by a costmap-based dynamic-obstacle tracker, as a QP at the controller’s output.
- C4 (reproducibility). An open Apache-2.0 implementation,¹ a standalone 2D mechanism validation, a

J. M. Kang is an independent researcher based in Seoul, Republic of Korea (e-mail: kangjmo91@gmail.com).

¹Code, benchmark scripts, and raw trial data: <https://github.com/kjungmo/se-mppi-nav2> (Apache-2.0); see the Appendix for the regeneration commands.

randomized 1,200-trial 2D benchmark with paired significance tests, and a specified full-scale ROS 2 physics-benchmark protocol.

We are deliberately conservative about novelty and about what we claim empirically: fusing CBFs with MPPI is not new. Our measured win is the online detection-and-escape layer (0% $\rightarrow$ 88–90% on the static U-trap family, Holm-adjusted $p = 1.4 \times 10^{-9}$, and 0% $\rightarrow$ 62–78% on the narrow-dynamic family, $p_{\text{adj}} = 1.1 \times 10^{-6}$, with the gap search load-bearing; Sec. VI-C). The escape–safety coordination (C2) is a certified-safe mechanism (Sec. IV-E) that, in the regime we could test, is empirically indistinguishable from an independent escape+CBF stack; we report this null openly and delimit where coordination should matter. The remaining load-bearing claims are Nav2-native deployment and the deployment faults a live stack exposes.

II. Related Work

MPPI and local controllers. Nav2 MPPI [1], [2], TEB [3], DWB (a Nav2 DWA-family planner) [4], and Regulated Pure Pursuit [5] are the standard deployable local planners. MPPI’s critic-plugin interface (`mppi::critics::CriticFunction`) exposes per-rollout trajectories, the reference path, the goal, collision flags, and the furthest-reached path index – i.e., the signals needed for entrapment detection are already available, and a custom critic can be inserted without forking the optimizer.

Local-minima escape. Most methods are always-on augmentations of the sampling distribution: log-MPPI [6], Tsallis-MPPI [7], SVG-MPPI (Stein mode-seeking) [8], Biased-MPPI [9], and learned sampling distributions for MPC (Sacks and Boots) [10]. The closest prior detect-and-switch method is DRPA-MPPI (Fuke et al., 2025 preprint) [11], which detects entrapment from predicted trajectories and switches on a repulsive-potential cost. DRPA-MPPI has no CBF or formal safety, models no dynamic obstacles, ships no public code, and is not Nav2-integrated. SE-MPPI uses it as the escape baseline.

CBF $\times$ MPPI safety. All four fusion styles exist as research code: soft cost/critic (Shield-MPPI [12], BR-MPPI [13]), output QP filter (CBFKit [14], the Shield stage), unsafe-sample rejection (reach-avoid SCBF-MPPI [15], DualGuard stage 1 [16]), and provably-safe projection/shielding (GS-MPPI [17], DualGuard [16]). For dynamic obstacles, DPCBF (“Beyond Collision Cones,” ICRA 2026) [18] uses a parabolic safe set that is far less conservative than collision-cone/velocity-obstacle CBFs and stays QP-feasible in dense dynamic scenes – but is QP-only and not embedded in MPPI. Critically, none of these ship as a Nav2 controller (CBFKit builds a standalone node).

Conformal prediction for safe control (background, not claimed). The released controller carries an online conformal margin that inflates the CBF effective radius under prediction uncertainty; this is an implementation detail deferred to follow-up work (Sec. VII, SE-Predict) and

TABLE I

Positioning against factors (a) Nav2-native plugin, (b) online local-minima escape, (c) dynamic-obstacle CBF safety filter, (d) escape–safety gain coordination. No reviewed prior system holds more than one; SE-MPPI is the only work at the four-way intersection.

System	(a)	(b)	(c)	(d)
DRPA-MPPI [11]	–	✓	–	–
Shield-/BR-MPPI [12], [13]	–	–	✓	–
CBFKit [14]	–	–	✓	–
SCBF-MPPI / DualGuard [15], [16]	–	–	✓	–
GS-MPPI [17]	–	–	✓	–
DPCBF [18]	–	–	✓	–
SE-MPPI (ours)	✓	✓	✓	✓

is not a contribution of this paper. That margin draws on the established line of conformal-prediction-for-safe-control work [19], [20], [21], [22], [23]; Paper 1’s claims are C1–C4 and stand without it.

Positioning. SE-MPPI occupies a four-way intersection that no single prior work holds, based on a June 2026 review of the Nav2-MPPI critic ecosystem, the CBF–MPPI fusion papers cited above, and the ROS Discourse / GitHub issue threads for these projects: (a) a Nav2-native plugin, (b) online local-minima escape, (c) a dynamic-obstacle CBF safety filter, and (d) escape–safety gain coordination. Table I tabulates the named systems against these four factors: no reviewed work holds more than one, and in particular no Nav2-native controller carries a CBF. Element (d) – the gain coordination – is the cleanest unoccupied contribution.

III. Problem Formulation

Consider a differential-drive robot with pose (x, y, θ) and control $u = (v, \omega)$, tracking a global path σ produced by a Nav2 global planner, amid static structure and a set $\mathcal{O}$ of dynamic obstacles o with position p_o , radius R_o , and (estimated) velocity v_o . Let r be the robot’s inscribed radius and m a safety margin. We want a control policy that (G1) makes monotone progress along σ – i.e., does not stall in a local minimum – and (G2) keeps the robot collision-free with respect to $\mathcal{O}$ with a forward-invariance guarantee, while (G3) running as a Nav2 controller at ≥ 10 Hz on CPU.

The tension: G1 may require maneuvers that approach obstacles (to round a trap), whereas a naive safety filter for G2 forbids exactly those maneuvers. SE-MPPI’s coordinator (Sec. IV-E) is the mechanism that satisfies G1 and G2 jointly.

IV. Method: SE-MPPI

A. Overview

SE-MPPI subclasses the stock `MPPIController`, reusing its optimizer for the nominal command, then post-processes that command each cycle:

- 1) Entrapment detection from monotone global-path progress (single source of truth, shared with the critic).

- 2) Escape at sampling time: an EscapeCritic injects distance-field APF and free-space gap-attraction costs only when entrapment is detected.
- 3) Dynamic-obstacle tracking from the local costmap.
- 4) Coordination: resolve the CBF gain α from entrapment and TTC.
- 5) CBF safety filter: project the nominal (v, ω) onto the CBF-safe set via a small QP.

The escape and safety layers thus act at complementary points – sampling-time cost shaping and output-time projection – unified by a shared entrapment signal and the coordinated gain.

Fig. 1 summarizes the resulting data flow.

B. Entrapment detection (single source of truth)

$\text{nearestPathIndex}(\sigma)$ is non-monotone, so we track the furthest reached path index $k_{\max}$ and declare entrapment when $k_{\max}$ fails to advance for a window of W cycles (default 30; the 2D benchmark of Sec. VI-C uses 15). Entrapment is suppressed within a goal tolerance so a robot finishing at the path end does not trigger a false escape. The controller is the only detector; the EscapeCritic and the coordinator read the same shared atomic state via a registry keyed by the controller name, so escape and safety always agree on whether the robot is trapped.

C. Escape (detect-and-switch)

On entrapment the EscapeCritic adds two cost terms to the MPPI rollouts:

- Distance-field APF $U = \frac{1}{2}\eta(1/d - 1/d_0)^2$ for $d < d_0$, where d is the Dijkstra distance to the nearest obstacle cell (excluding unknown space), which pushes rollouts off the trapping surface; and
- Free-space gap attraction: a ray-cast around the robot finds the opening whose bearing is closest to the goal direction and whose clearance exceeds a threshold, and rollouts heading toward that gap are rewarded – providing a temporary subgoal that rounds non-convex (U-shaped) traps.

Because the terms are injected only when entrapped, free-space behavior is unchanged (detect-and-switch, not always-on).

D. CBF safety filter (dynamic obstacles)

To make the barrier relative-degree one in (v, ω) for a unicycle, safety is enforced on a look-ahead point $P = (x, y) + L(\cos \theta, \sin \theta)$, which is fully actuated by (v, ω) through the Jacobian

$$G = \begin{bmatrix} \cos \theta & -L \sin \theta \\ \sin \theta & L \cos \theta \end{bmatrix}. \quad (1)$$

For each obstacle o with $d = P - p_o$ and effective radius $R = r + R_o + m$:

$$h_o = \|d\|^2 - R^2, \quad \dot{h}_o = 2d^\top (Gu - v_o). \quad (2)$$

The continuous-time CBF condition $\dot{h}_o + \alpha h_o \geq 0$ [24], [25], enforced once per control cycle, is linear in u . We solve a small QP (OSQP) [26] that minimizes

$$\|u - u_{\text{nom}}\|^2 + \rho \delta^2 \quad (3)$$

subject to the per-obstacle CBF rows (relaxed by a single slack $\delta \geq 0$ for feasibility) and the input box limits. Obstacles are pruned to the nearest few by clearance (surface gap), not center distance. If the QP can only stay safe by relaxing the barrier ($\delta > \epsilon$) or fails, the controller brakes the forward velocity to zero while keeping the QP's turn (or, on solver failure, the box-clamped nominal turn) – stop rather than drive into an imminent collision. Static structure is intentionally excluded from the CBF (handled by the MPPI obstacle critic and costmap inflation); only obstacles that are moving and small enough to be a movable body are admitted, which we found essential in a real costmap (Sec. VI-B).

Because this condition is checked once per control cycle (a sampled-data CBF), strict between-sample forward invariance would require a sampled-data margin tightening R by an $O(\dot{h}_{\max} \Delta t)$ term; in practice the obstacle-radius margin m absorbs this gap, and we report the empirical safety rate rather than claim continuous-time invariance between samples.

E. Escape–safety coordination (the contribution) and forward invariance

The coordinator resolves the gain:

$$\alpha = \begin{cases} \alpha_{\text{base}} & \text{not entrapped, or TTC} < \tau \text{ (imminent)} \\ \alpha_{\text{escape}} & \text{entrapped and TTC} \geq \tau \end{cases} \quad (4)$$

with $\alpha_{\text{escape}} > \alpha_{\text{base}} > 0$, where TTC is the minimum time-to-collision over tracked obstacles under the current command.² Raising α relaxes the constraint $\dot{h} \geq -\alpha h$ – permitting the robot to approach an obstacle faster, as an escape maneuver requires – while the TTC override snaps the gain back to α_{base} (safety first) when a dynamic obstacle is imminent.

Proposition (certified-safe escape). Fix an obstacle j , suppose the safety-filter QP is feasible without slack throughout the interval considered (the shared $\delta(t) = 0$), and assume the continuous-time inequality $\dot{h}_j(t) \geq -\alpha(t) h_j(t)$ holds for almost every $t \geq 0$ – an assumption between samples, since discrete-instant QP feasibility does not by itself imply the intersample inequality (the sampled-data caveat of Sec. IV-D). Let $\alpha : [0, \infty) \rightarrow [\alpha_{\min}, \alpha_{\max}] \subset (0, \infty)$ be any measurable, positive, bounded gain schedule – in particular the coordinator's piecewise-constant switching between α_{base} and α_{escape} (Eq. 4) and its TTC override.

²The released controller's coordinator additionally holds α at α_{base} whenever the online conformal bound q on obstacle-prediction error exceeds a trust threshold (0.25 m by default; the calibrator is enabled by default), and that bound inflates the effective radius R of Sec. IV-D – the SE-Predict margin noted in Sec. II, not claimed by this paper. An implementation following Eq. (4) alone therefore reproduces the benchmark configurations of Sec. VI-C, not the released defaults.

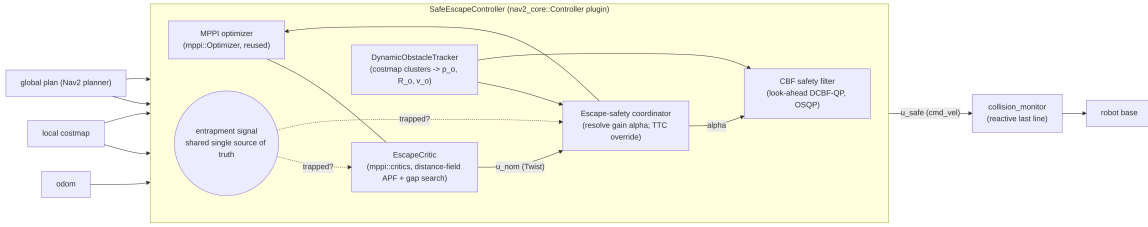


Fig. 1. SE-MPPI architecture / data flow. Global plan, local costmap, and odometry feed the SafeEscapeController plugin: the reused MPPI optimizer (with the EscapeCritic) emits the nominal command, a shared entrapment signal drives both the critic and the escape-safety coordinator, the coordinator resolves the CBF gain α (with TTC override), and the CBF safety filter – fed by the DynamicObstacleTracker – projects the command to `cmd_vel`, which Nav2’s `collision_monitor` guards as a final reactive layer.

If $h_j(0) \geq 0$, then $h_j(t) \geq 0$ for all $t \geq 0$ – the look-ahead point P never enters obstacle j ’s disc (body safety approximate; see Sec. VII).

Proof. Define the integrating factor $\varphi(t) = \exp(\int_0^t \alpha(s) ds)$, finite and strictly positive because $\alpha \leq \alpha_{\max}$. Then, almost everywhere,

$$\frac{d}{dt} [\varphi(t) h_j(t)] = \varphi(t) [\dot{h}_j(t) + \alpha(t) h_j(t)] \geq 0, \quad (5)$$

so $\varphi(t) h_j(t)$ is non-decreasing, hence $\varphi(t) h_j(t) \geq \varphi(0) h_j(0) = h_j(0) \geq 0$. Since $\varphi(t) > 0$, $h_j(t) \geq 0$ for all t . ■

Corollary. Forward invariance of $\mathcal{C}_j = \{h_j \geq 0\}$ holds for every positive, bounded gain schedule – in particular for the coordinator’s switching between α_{base} and α_{escape} , and for the TTC override that drops α back to α_{base} . Raising α never permits $h_j < 0$; it only weakens the lower bound on $\dot{h}_j$, letting the robot approach $\partial\mathcal{C}_j$ faster (bolder detours). Thus escape aggressiveness and the safety certificate are decoupled: the gain trades conservativeness for maneuverability within an invariant safe set. When the QP returns $\delta = 0$ on all rows, every h_j satisfies the hypothesis simultaneously, so $\mathcal{C} = \bigcap_j \mathcal{C}_j$ is invariant; the argument is per-obstacle and gain-agnostic, so sharing a single $\alpha(t)$ across rows is immaterial to the invariance statement itself.

This is why coordination, not stacking, is the design we adopt: the two layers share one invariant set, and the gain is the single knob that trades exploration for caution inside it – a certificate property that holds whether or not the barrier binds (Sec. VI-C1). The guarantee is conditional on $\delta = 0$ and on the obstacle model; the shared slack means one imminent obstacle can momentarily loosen the margin on every other obstacle in the same QP solve, which is why we report the empirical slack-usage rate rather than an unconditional certificate: in the 2D benchmark of Sec. VI-C the QP never engages slack on the mover-free families, and uses nonzero slack in 26–34 of 50 narrow-dynamic trials (max 0.47) and 47–48 of 50 dynamic-family trials (max 0.015) per CBF-carrying config (Sec. VI-C1). The $\delta > 0$ brake is the safe fallback, and bounded-prediction-error robustness is the subject of follow-up work (Sec. VII).

F. Dynamic-obstacle tracking

The tracker clusters costmap cells above a lethal threshold, associates each current cluster to at most one previous cluster within a gate (so a split/merge cannot manufacture a phantom velocity), and estimates a clamped constant velocity. Its output $\{p_o, R_o, v_o\}$ feeds the coordinator (TTC) and the CBF filter. The constant-velocity model is a deliberate, replaceable baseline (Sec. VII).

V. Implementation (Nav2-Native)

SE-MPPI is built on ROS 2 Jazzy + Nav2 [1]. The package `nav2_se_controller` ships three shared libraries: a plugin-free `se_mppi_core` (entrapment, CBF, coordinator, tracker, repulsion, gap search), the `escape_critic` MPPI critic, and the `safe_escape_controller`. Separating the plugin-free core from the two plugin libraries resolves a class_loader factory conflict that otherwise prevents the critic and controller from co-loading.

Two integration details proved essential and are easy to get wrong:

- Critic namespace. Nav2’s CriticManager resolves a critics-list entry NAME to the plugin class `mppi::critics::NAME`. A critic in any other namespace can be unit-tested via pluginlib directly but will never load from the critics list. EscapeCritic therefore lives in `mppi::critics`.
- Jazzy critic data uses `xtensor`, not the Eigen of the main branch; the critic was written against the installed headers.

The QP uses `osqp-eigen` [26]. The build is reproducible via RoboStack (conda-forge) so the build does not depend on the host OS version (validated on Ubuntu 20.04 with an NVIDIA GPU). The package’s Paper-1 modules (entrapment detector, CBF filter, coordinator, tracker, gap search, repulsion, path progress, and plugin loading) carry 44 unit tests (gtest), plus linters; the wider package additionally carries tests for the follow-up papers’ prediction stack (static/dynamic occupancy classification, trajectory prediction, conformal calibration) and multi-robot coordination module (SE-Predict, Multi-SE-MPPI), plus the live-injection ABI regression of Sec. VI-B, bringing the current suite to 82 TEST cases across 13 files.

VI. Experiments

A. Mechanism validation (standalone 2D)

To validate each mechanism independently of simulator infrastructure, we reimplement the controller’s core math (APF, gap raycast, look-ahead CBF-QP via OSQP [26], α modulation + TTC override, monotone-progress entrapment) as a 2D unicycle and run three scenarios. The 2D code mirrors the C++ formulas one-to-one. Measured results:

Three findings: (1) stock MPPI stalls in front of a U-trap while SE-MPPI detects and rounds it; (2) against a crossing dynamic obstacle, the no-CBF baseline collides while the speed-aware CBF filter avoids it; and (3) the coordination plot shows the gain rise $\alpha : 2 \rightarrow 6$ during the escape phase with slack ≈ 0 throughout – i.e., the escape maneuver is admitted and certified-safe, directly demonstrating the Proposition of Sec. IV-E. In these benign scenarios the coordinated and independent variants are both safe; whether coordination yields a quantitative outcome benefit is tested directly in the randomized benchmark of Sec. VI-C, which finds it null in the regimes reachable there (Sec. VI-C1).

These three findings are shown in Fig. 2 (sources in experiments/prototype/figures/; generated by experiments/prototype/run_validation.py).

B. Live Nav2 + Gazebo integration

We deployed SE-MPPI in a complete live stack (ROS 2 Jazzy, Nav2 – AMCL, costmaps, NavFn, BT navigator, recovery, collision monitor – and Gazebo tb3_sandbox) [1] on the target workstation. Confirmed at the level of committed launch logs and trial telemetry: the controller and EscapeCritic load and activate from the critics list; the controller produces valid velocity commands identical in form to stock MPPI; and the full lifecycle reaches the active state, sustaining multi-minute drives on a U-trap testbed without a crash or collision after the ABI fix below (the goal was not reached: the committed telemetry records STUCK at 254.5 s and TIMEOUT at 299.8 s, both collision-free, under the host constraints noted below). The escape-injection path has also run live: its first live activation is what exposed the ABI fault below – a fault unreachable from unit tests – and the tensor shapes captured from that event are committed as a regression test, while the controller and critic now log every detection and injection transition (ENTER/EXIT, BEGIN/END) for auditability. A sustained live entrapment-escape demonstration on this host remains confounded by the host-level artifacts noted below and is deferred to the full-scale benchmark. We validate the live integration to this depth; the controlled quantitative comparison is the 2D benchmark of Sec. VI-C, because a full live A/B on this WSL2 host measures the host (RTF ≈ 0.3 – 0.6 , with TF smear inflating the costmap with phantom cells) rather than the algorithm.

Three integration faults surfaced that only a live stack reveals, each reported as a deployment lesson:

- Static walls in the CBF. Feeding every lethal cluster (including static walls) into the CBF makes the barrier infeasible everywhere and freezes the robot – fixed by scoping the CBF to genuinely dynamic obstacles (Sec. IV-D).
- Compute budget. The per-cycle budget must be matched to the host (horizon preserved, rate and batch reduced) or the loop misses its rate and the optimizer diverges.
- xtensor ABI mismatch. The installed libmppi_controller.so is compiled with XTENSOR_USE_XSIMD + AVX2 (an xsimd::aligned_allocator element type), whereas our plugin compiled xtensor with the default std::allocator. Because CriticData’s tensors cross the pluginlib boundary by reference, the first EscapeCritic write to data.costs was an ODR violation that segfaulted the entire Nav2 process on the first live escape injection (3/3 reproducible). The in-process unit tests never caught it – they compile every translation unit with one consistent flag set – and earlier live gates never triggered entrapment, so the injection path had never actually run live. Fixed by compiling the critic against the installed headers’ flags (per-target, since a global AVX2 flip breaks the separately built OSQP–Eigen boundary); a regression test now replays the injection path with live tensor shapes. This is precisely the class of fault a Nav2-native deployment exposes and that a standalone node or a pure-2D study cannot.

C. Randomized 2D quantitative benchmark

The controlled quantitative comparison is a randomized 2D benchmark that mirrors the controller math one-to-one. It reuses the exact primitives validated in Sec. VI-A – the same MPPI sampler, distance-field APF, gap raycast, look-ahead CBF-QP via OSQP [26], α coordination with TTC override, and monotone-progress entrapment detector – and drives them across randomized scenarios so that success, collision, and clearance carry confidence intervals and paired significance tests. It is not the live Nav2 controller and inherits the scope limits of Sec. VI-A (a 2D unicycle, a free-space gap subgoal in place of global routing, and a look-ahead-point certificate rather than full-body); as in the released controller and the deployment lesson of Sec. VI-B, the CBF and TTC see only the dynamic obstacles, not static walls. We use this 2D surface rather than a live Gazebo full matrix because at RTF 0.3–0.6 under CPU contention the live host measures itself, not the algorithm (Sec. VI-B); the full-scale physics benchmark is deferred to future work (Sec. VII).

Setup. Four seed-deterministic scenario families exercise the two weaknesses and their conjunction: utrap (a finite U/C pocket with the goal behind the back wall – a static local minimum), clutter (a random field of 5–9 circular obstacles with a guaranteed footprint-clear path), dynamic (1–2 pedestrian-like crossing movers timed

TABLE II
Standalone 2D mechanism validation (measured).

Scenario / config	reached	collided	time	min-clear (m)
U-trap / Stock MPPI	× (stuck)	—	—	0.33
U-trap / SE-MPPI (escape)	✓	—	27.7 s	0.32
Dynamic / No CBF	×	collision	—	−0.00
Dynamic / SE-MPPI (CBF)	✓	safe	18.6 s	0.01
Coordination: SE-MPPI vs. independent [†]	✓	safe	27.7 s	0.32

[†]SE-MPPI (F) and the independent stack (E) return identical trajectories and metrics in this benign scenario; the quantitative null test is Table IV.

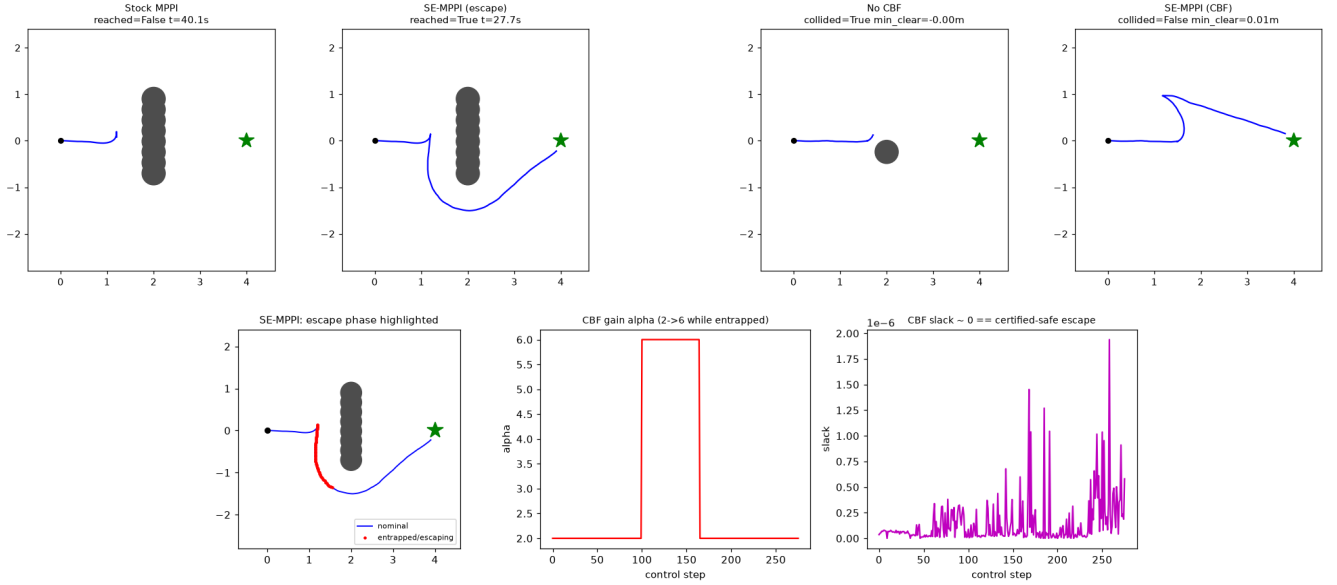


Fig. 2. Standalone 2D mechanism validation (trajectories/traces only; no benchmark metrics). (a) U-trap escape: stock MPPI (left) stalls at the trap mouth ($x \approx 1.2$, no progress) while SE-MPPI (right) detects the stalled progress and rounds the U-shaped obstacle to the goal via the gap-attraction subgoal. (b) Dynamic-obstacle avoidance: against a crossing obstacle the no-CBF baseline (left) collides while the speed-aware look-ahead CBF filter (right) projects the command to a collision-free path. (c) Escape-safety coordination (the contribution): during the escape phase the coordinated gain rises $\alpha : 2 \rightarrow 6$ while the QP slack stays ≈ 0 throughout – the escape maneuver is admitted yet remains certified-safe ($h \geq 0$), the mechanical evidence for the Proposition of Sec. IV-E.

to reach the robot's path), and narrowdyn (the utrap pocket plus movers sweeping the pocket mouth during the escape window – the narrow-and-dynamic coincidence the coordination targets). Six ablation configs toggle the mechanisms: A stock (no escape/CBF), C escape only, D CBF only, E escape+CBF with $\alpha_{\text{escape}} = \alpha_{\text{base}}$ (independent), F full SE-MPPI ($\alpha : 2 \rightarrow 6$ + TTC override, coordinated), and F[−] (F with the gap search off). Each family is a pure function of an integer seed, so every config is evaluated on an identical scenario set, as the paired McNemar design requires: 4 families $\times$ 50 seeds $\times$ 6 configs = 1,200 paired trials, 40 s sim budget each. Success carries a Wilson 95% CI; the primary test is McNemar on paired outcomes; continuous metrics use Mann–Whitney U with Cliff's δ ; all p -values are Holm–Bonferroni-corrected within each family.

The escape layer is the measured contribution, and the gap search is load-bearing. On both trap families the effect is decisive and survives Holm correction (Table IV): success rises from 0% (stock A, CBF-only D, and no-gap F[−] all time out in the pocket) to 88–90% on utrap

TABLE III
Randomized 2D benchmark: success rate in % (Wilson 95% CI), $N = 50$ per cell. On the trap families (utrap, narrowdyn) the escape configs C, E, F separate from stock A, CBF-only D, and no-gap F[−]; F[−] collapses to 0%; E and F coincide except on narrow-dyn, where they differ by one trial (64% vs 62%).

Config	U-trap	Clutter	Dynamic	Narrow-dyn
A stock	0 [0,7]	52 [39,65]	80 [67,89]	0 [0,7]
C escape	90 [79,96]	62 [48,74]	76 [63,86]	78 [65,87]
D CBF	0 [0,7]	52 [39,65]	82 [69,90]	0 [0,7]
E indep.	88 [76,94]	62 [48,74]	78 [65,87]	64 [50,76]
F SE-MPPI	88 [76,94]	62 [48,74]	78 [65,87]	62 [48,74]
F [−] no-gap	0 [0,7]	52 [39,65]	84 [71,92]	0 [0,7]

(44/50 discordant pairs, $p_{\text{adj}} = 1.4 \times 10^{-9}$) and to 62–78% on narrowdyn (31/50 discordant, $p_{\text{adj}} = 1.1 \times 10^{-6}$). That F[−] scores 0% on both trap families (Table III) shows the detect-and-switch mechanism escapes only with the free-space gap subgoal: APF and gain modulation without a routed opening do not escape. On clutter the escape adds 10 pp (52% $\rightarrow$ 62%, not significant after Holm) at the cost

TABLE IV

Key paired contrasts (McNemar, Holm-adjusted; b/c discordant pairs, b = baseline wins, c = F wins). The escape contrast is decisive on the trap families; the coordination contrast ($E \rightarrow F$) is null everywhere.

Family	Escape (stock $\rightarrow$ F)		Coord. ($E \rightarrow F$)	
	b/c	p_{adj}	b/c	p_{adj}
U-trap	0/44	$1.4 \times 10^{-9} *$	0/0	1
Clutter	0/5	1	0/0	1
Dynamic	3/2	1	0/0	1
Narrow-dyn	0/31	$1.1 \times 10^{-6} *$	1/0	1

of ≈ 2.4 s longer average time-to-goal (over successful trials), and on the pure-dynamic family – which contains no trap – it is appropriately neutral (Table IV, dynamic stock $\rightarrow$ F: $p = 1$). Fig. 3 gives the full per-family picture.

1) Escape-safety coordination is null in this regime: The load-bearing contrast for the C2 contribution is E (independent) vs. F (coordinated): identical mechanisms differing only in whether the CBF gain is coordinated with the escape phase. Across all 200 paired scenarios this contrast is null (Table IV, Fig. 4): the discordant counts are 0/0, 0/0, 0/0, and 1/0, McNemar $p = 1$ in every family, and every continuous effect size is $|\delta| \leq 0.03$. The gain does engage as designed ($\alpha_{\text{max}} = 6$ during escape, TTC override active). The committed per-trial slack telemetry (slack_max in trials.csv; per-config rates in stats.json and tables.md) locates where the barrier actually works. On the mover-free families (utrap, clutter) the dynamic-obstacle CBF sees no obstacles at all: slack is zero in every trial, E and F issue identical commands, and the null there is structural. On narrowdyn the barrier does engage and even relaxes – the QP uses nonzero slack in 34/50 trials for E and F (26/50 for D and F $^-$; max slack 0.47) – and on dynamic in 47–48/50 trials (max 0.015), yet the E-vs-F contrast stays null (1/0 discordant, $p = 1$). The correct reading is therefore not that the barrier never binds, but that the outcomes are insensitive to the α schedule: where the barrier is quiescent the two configs coincide exactly, and where it is active the base and escape gains admit escape maneuvers whose outcomes are indistinguishable at $N = 50$. (The mechanism plot of Sec. VI-A, “ $\alpha : 2 \rightarrow 6$ with slack ≈ 0 ,” shows the benign end of this spectrum.) We report this honestly: in this 2D benchmark the coordination produces no measurable outcome difference over independent escape+CBF. The result is consistent with – and delimits – the Proposition of Sec. IV-E: coordination is a certified-safe mechanism (the escape maneuver is admitted while $h \geq 0$ is preserved), not, in these scenarios, an outcome improvement. An outcome-level benefit would require a regime where the choice of α changes which escape maneuvers are admitted at outcome-deciding moments – faster or persistent movers, tighter margins, a dynamic gap the escape route must thread while the barrier is live; this generator’s pedestrian-like movers produce barrier engagement but not that sensitivity. Constructing and measuring that

regime is future work (Sec. VII); we claim no coordination benefit we have not measured.

Two CBF observations bear on Sec. VII. Against body-grazing crossers (dynamic) the look-ahead-point filter leaves the collision rate essentially unchanged (20% $\rightarrow$ 16–18%; descriptive – the committed analysis carries no paired collision-outcome test). And in the cramped narrowdyn pocket the CBF configs show a higher collision rate than their no-CBF counterparts (18% $\rightarrow$ 32%, descriptive) and cost ≈ 16 pp success (78% $\rightarrow$ 62%; success McNemar F vs. C: $b/c = 8/0$, raw $p = 0.008$, Holm-adj. 0.109 – directionally consistent though not significant after correction at $N = 50$). Both are discussed as limitations in Sec. VII.

Full-scale benchmark (future work). The live full-matrix comparison this section stands in for is specified but deferred: an identical Nav2 stack and differential robot across three tiers – BARN (static difficulty) [27], [28], DynaBARN (dynamic) [29], and HuNavSim (human-aware) [30] – with stock MPPI [1], [2], DWB [4], RPP [5], TEB [3], an always-on escape (DRPA-style [11]), and a CBF-only baseline (Shield-style [12]), scored by success, collision, time-to-goal, path length, clearance, and per-cycle compute under the same McNemar / Mann-Whitney + Holm protocol. Running it on a real-GPU host is the remaining step (Sec. VII).

VII. Limitations

- Full-scale physics benchmark is future work. The quantitative comparison in this paper is the randomized 2D benchmark of Sec. VI-C, which mirrors the controller math but is not the live controller. The full-scale 3D physics benchmark specified there (BARN/DynaBARN/HuNavSim against the deployable baselines) is not yet run: the live host available to us (WSL2, RTF ≈ 0.3 – 0.6 , with TF smear inflating the costmap with phantom cells) measures itself rather than the controller (Sec. VI-B), so it needs a real-GPU host.
- Coordination benefit is unmeasured, not established. The escape-safety coordination (C2) is certified-safe by construction (Sec. IV-E) but produced no outcome difference over an independent escape+CBF stack: outcomes proved insensitive to the α schedule even where the barrier engages (Sec. VI-C1). Demonstrating an outcome-level benefit requires a regime where the α choice decides which escapes are admitted (faster/persistent movers, tighter dynamic gaps); constructing and measuring it is future work.
- Braking is not a safe fallback when cornered. In the cramped narrow-dynamic pocket the infeasible-QP fallback ($v = 0$) leaves the robot in the path of a sweeping mover, and the CBF configs measured a higher collision rate than their no-CBF counterparts (18% $\rightarrow$ 32%; Sec. VI-C). A cornered robot needs an evasive fallback, not a stop.
- Look-ahead-point vs. full-body safety. The certificate of Sec. IV-E is for the look-ahead point P

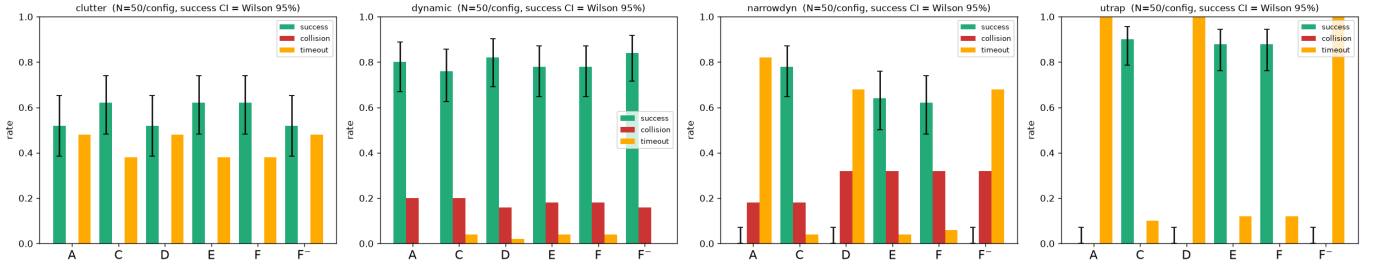


Fig. 3. Randomized 2D benchmark: success / collision / timeout per family across configs A–F[−] (Wilson 95% error bars on success, $N = 50$). On the trap families (utrap, narrowdyn) the escape configs (C = escape-only, E = independent escape+CBF, F = coordinated) separate cleanly from stock A, CBF-only D, and no-gap F[−], which time out at 0% success. E and F coincide in every family except narrow-dynamic, where they differ by a single discordant trial (McNemar $p = 1$).

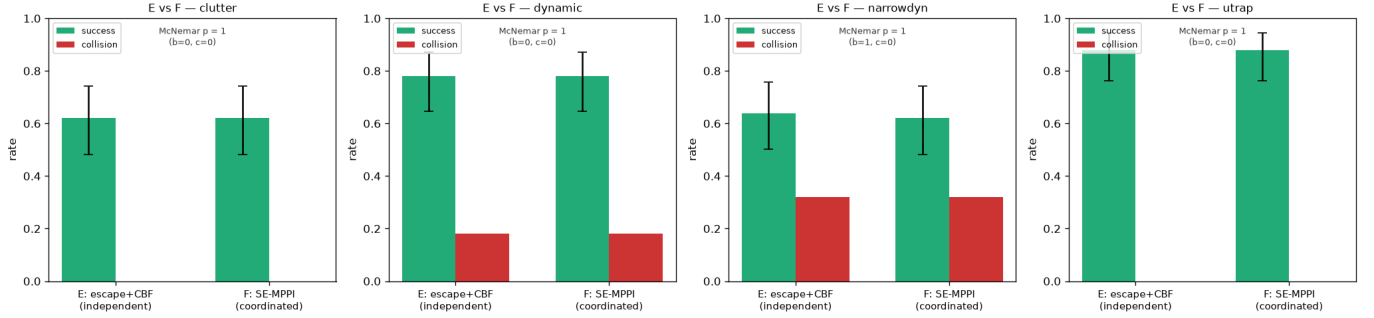


Fig. 4. The coordination contrast E vs. F per family, annotated with the McNemar p -value. The independent (E) and coordinated (F) stacks are statistically indistinguishable in every family ($p = 1$) – the null result of Sec. VI-C1.

(the relative-degree-one output), not the full robot body. Body safety is therefore approximate: the offset $L \approx r$ places P at the robot’s leading edge, and the costmap obstacle critic plus collision_monitor cover the residual. Full-body certification would require inflating the margin by L , which we deliberately avoid because it over-constrains sub-meter gaps (BARN’s maximum clearance is ≈ 0.9 m) and would sharply degrade the success rate. This gap is measured: on the dynamic family the look-ahead filter left body-grazing collisions essentially unchanged (20% $\rightarrow$ 16–18%, descriptive; Sec. VI-C).

- Constant-velocity tracker (addressed in the released controller). The dynamic-obstacle model evaluated in this paper is a replaceable constant-velocity (CV) baseline with rotating/accelerating agents mispredicted and prediction uncertainty unquantified. The released controller now ships (i) occupancy-persistence static/dynamic classification (no wall-freeze on association jitter), (ii) persistent tracks with least-squares CV/CVCA horizon prediction, and (iii) an online conformal bound q that inflates the CBF effective radius (time-varying radius) and gates the escape gain on prediction trust. The forward-invariance argument is therefore conditional only on the bounded prediction error that the calibrator maintains, not on model correctness. These additions are the subject of the follow-up paper (SE-Predict); this paper’s claims are isolated from them, and the

follow-up paper’s own no-conformal ablation will isolate them there.

- TTC is a 1-D approximation; tight spaces can induce over-rotation.
- Real-time. Per-call QP cost grows with the obstacle budget (capped by clearance pruning).
- Localization in symmetric/narrow maps can jump and perturb progress estimation – an environment property, mitigated in structured benchmarks.

VIII. Conclusion

SE-MPPI packages local-minima escape and control-barrier-function safety, usually studied apart, in a single deployable Nav2 controller: a shared entrapment signal drives both a sampling-time escape critic and an output-time CBF filter, and a coordinated gain lets the robot escape while a forward-invariance argument keeps the escape certified-safe. A randomized 1,200-trial 2D benchmark shows the online detection-and-escape layer is the decisive contribution – 0% to 88–90% success on the static U-trap family and 0% to 62–78% on the narrow-dynamic family, with the free-space gap search load-bearing – while the escape-safety coordination, certified-safe by construction, is empirically indistinguishable from an independent escape+CBF stack in the regimes we could test (the null of Sec. VI-C1), which we report openly. We fixed three integration faults that only a live stack reveals – static-wall CBF infeasibility, host-matched compute budgeting, and an xtensor ABI mismatch that segfaulted Nav2 on the first

live escape injection – and confirmed the controller loads, activates, and runs in a full ROS 2 Jazzy + Nav2 + Gazebo deployment, with the live injection event that exposed the ABI fault preserved as a committed regression test. The remaining step is a full-scale 3D physics benchmark on a real-GPU host. Beyond this paper, the codebase has grown the uncertainty-calibrated prediction stack (SE-Predict: classification, horizons, conformal bounds – paper 2) and the multi-robot reciprocal coordination layer (Multi-SE-MPPI – paper 3); both reuse this paper’s coordination thesis and the shared evaluation harness.

Appendix Reproducibility

All paths below are relative to the code release of contribution C4 (footnote in Sec. I).

Software. ROS 2 Jazzy, Nav2, and Gazebo are installed via RoboStack (conda channels robostack-jazzy + conda-forge) by the committed script `scripts/setup_ros2_env.sh`, which also pulls `osqp-eigen/eigen` for the CBF-QP [26] and the `cxx-compiler/cmake/ninja` toolchain, so the build does not depend on the host OS version (Sec. V).

Benchmark regeneration. Every number in Sec. VI-C is generated from the committed per-trial CSV by committed scripts, never transcribed by hand. The commands (from `docs/papers/2026_2d-benchmark-results.md`, re-wrapped to column width; the committed data uses $N=50$):

```
export MAMBA_ROOT_PREFIX=$HOME/micromamba
micromamba run -n ros2 python3 \
  -m experiments.benchmark2d.runner \
  --families utrap clutter dynamic narrowdyn \
  -n <N> --workers 6 --max-steps 400
micromamba run -n ros2 python3 \
  -m experiments.benchmark2d.report # tables + stats
micromamba run -n ros2 python3 -c \
  "from experiments.benchmark2d import figures, \
  aggregate; \
  r=aggregate.aggregate( \
    'experiments/results_2d/trials.csv', \
    'experiments/results_2d'); \
  figures.plot_all(r['summary'], r['stats'], \
    'experiments/results_2d/figures')"
```

Scenarios and seeds. The benchmark is 4 families $\times$ 50 seeds $\times$ 6 configs = 1,200 paired trials (seeds 0–49 per family, 400 steps $\times$ 0.1 s budget each). Each scenario is a pure function of its integer seed: the generators in `experiments/benchmark2d/scenarios.py` seed numpy generators with a deterministic per-family offset plus the trial seed (10^6 utrap, 2×10^6 clutter, 3×10^6 dynamic, 4×10^6 narrowdyn), and the MPPI noise stream uses the same trial seed (`experiments/benchmark2d/runner.py`), so every config sees byte-identical worlds and rollout noise – the common-random-numbers setup the paired McNemar design requires. The six config toggle sets are `experiments/benchmark2d/configs.py`; the ablation overlays for the deferred live full-scale protocol are `experiments/configs/ablations.yaml`.

Data and parameters. Raw per-trial data: `experiments/results_2d/trials.csv`; aggregates and statistics: `summary.csv`, `stats.json`, `tables.md` (same

directory, all auto-generated). The full released-controller parameter set is `src/nav2_se_controller/config/nav2_se_controller_params.yaml`.

References

- [1] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, “The Marathon 2: A Navigation System,” in 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020. [Online]. Available: <https://arxiv.org/abs/2003.00368>
- [2] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Information Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.02342>
- [3] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, “Trajectory Modification Considering Dynamic Constraints of Autonomous Robots,” in Proceedings of ROBOTIK 2012; 7th German Conference on Robotics. VDE Verlag, 2012, pp. 74–79. [Online]. Available: <https://ieeexplore.ieee.org/document/6309484>
- [4] D. Fox, W. Burgard, and S. Thrun, “The Dynamic Window Approach to Collision Avoidance,” IEEE Robotics & Automation Magazine, vol. 4, no. 1, pp. 23–33, 1997.
- [5] S. Macenski, S. Singh, F. Martin, and J. Gines, “Regulated Pure Pursuit for Robot Path Tracking,” Autonomous Robots, 2023. [Online]. Available: <https://arxiv.org/abs/2305.20026>
- [6] I. S. Mohamed, K. Yin, and L. Liu, “Autonomous Navigation of AGVs in Unknown Cluttered Environments: log-MPPI Control Strategy,” IEEE Robotics and Automation Letters, 2022. [Online]. Available: <https://arxiv.org/abs/2203.16599>
- [7] Z. Wang, O. So, J. Gibson, B. Vlahov, M. S. Gandhi, G.-H. Liu, and E. A. Theodorou, “Variational Inference MPC using Tsallis Divergence,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.00241>
- [8] K. Honda, N. Akai, K. Suzuki, M. Aoki, H. Hosogaya, H. Okuda, and T. Suzuki, “Stein Variational Guided Model Predictive Path Integral Control: Proposal and Experiments with Fast Maneuvering Vehicles,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.11040>
- [9] E. Trevisan and J. Alonso-Mora, “Biased-MPPI: Informing Sampling-Based Model Predictive Control by Fusing Ancillary Controllers,” IEEE Robotics and Automation Letters, 2024, arXiv:2401.09241. [Online]. Available: <https://arxiv.org/abs/2401.09241>
- [10] J. Sacks and B. Boots, “Learning Sampling Distributions for Model Predictive Control,” in Proceedings of the 6th Conference on Robot Learning (CoRL), 2022. [Online]. Available: <https://arxiv.org/abs/2212.02587>
- [11] T. Fuke, M. Endo, K. Honda, and G. Ishigami, “DRPA-MPPI: Dynamic Repulsive Potential Augmented MPPI for Reactive Navigation in Unstructured Environments,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.20134>
- [12] J. Yin, C. Dawson, C. Fan, and P. Tsiotras, “Shield Model Predictive Path Integral: A Computationally Efficient Robust MPC Approach Using Control Barrier Functions,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.11719>
- [13] H. Parwana, T. Kim, K. Long, B. Hoxha, H. Okamoto, G. Fainekos, and D. Panagou, “BR-MPPI: Barrier Rate Guided MPPI for Enforcing Multiple Inequality Constraints with Learned Signed Distance Field,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.07325>
- [14] M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and D. Prokhorov, “CBFKit: A Control Barrier Function Toolbox for Robotics Applications,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.07158>
- [15] H. Parwana, M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and D. Prokhorov, “Model Predictive Path Integral Methods with Reach-Avoid Tasks and Control Barrier Functions,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.13693>
- [16] J. Borquez, L. Raus, Y. U. Ciftci, and S. Bansal, “DualGuard MPPI: Safe and Performant Optimal Control by Combining Sampling-Based MPC and Hamilton-Jacobi Reachability,” IEEE Robotics and Automation Letters, vol. 10, no. 7, 2025. [Online]. Available: <https://arxiv.org/abs/2502.01924>

- [17] P. Rabiee and J. B. Hoagg, “Guaranteed-Safe MPPI Through Composite Control Barrier Functions for Efficient Sampling in Multi-Constrained Robotic Systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.02154>
- [18] H. K. Park, T. Kim, and D. Panagou, “Beyond Collision Cones: Dynamic Obstacle Avoidance for Nonholonomic Robots via Dynamic Parabolic Control Barrier Functions,” 2025, accepted to IEEE International Conference on Robotics and Automation (ICRA) 2026. [Online]. Available: <https://arxiv.org/abs/2510.01402>
- [19] H. Zhou, Y. Zhang, and W. Luo, “Safety-Critical Control with Uncertainty Quantification using Adaptive Conformal Prediction,” in 2024 American Control Conference (ACC), 2024. [Online]. Available: <https://arxiv.org/abs/2407.03569>
- [20] J. Gonzales, K. Mizuta, K. Leung, and L. J. Ratliff, “Safe Probabilistic Planning for Human-Robot Interaction using Conformal Risk Control,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.10392>
- [21] L. Lindemann, M. Cleaveland, G. Shim, and G. J. Pappas, “Safe Planning in Dynamic Environments using Conformal Prediction,” 2022. [Online]. Available: <https://arxiv.org/abs/2210.10254>
- [22] A. Dixit, L. Lindemann, S. Wei, M. Cleaveland, G. J. Pappas, and J. W. Burdick, “Adaptive Conformal Prediction for Motion Planning among Dynamic Agents,” 2022. [Online]. Available: <https://arxiv.org/abs/2212.00278>
- [23] L. Busellato, F. Cunico, D. Dall’Alba, M. Emporio, A. Giachetti, R. Muradore, and M. Cristani, “Uncertainty Aware-Predictive Control Barrier Functions: Safer Human Robot Interaction through Probabilistic Motion Forecasting,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.20812>
- [24] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control Barrier Function Based Quadratic Programs for Safety Critical Systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017. [Online]. Available: <https://doi.org/10.1109/TAC.2016.2638961>
- [25] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control Barrier Functions: Theory and Applications,” in 2019 18th European Control Conference (ECC), 2019, pp. 3420–3431. [Online]. Available: <https://arxiv.org/abs/1903.11199>
- [26] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: An Operator Splitting Solver for Quadratic Programs,” *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020. [Online]. Available: <https://arxiv.org/abs/1711.08013>
- [27] D. Perille, A. Truong, X. Xiao, and P. Stone, “Benchmarking Metric Ground Navigation,” 2020. [Online]. Available: <https://arxiv.org/abs/2008.13315>
- [28] X. Xiao, Z. Xu, A. Datar, G. Warnell, P. Stone, J. J. Damanik, J. Jung, C. A. Deresa, T. D. Huy, C. Jinyu, C. Yichen, J. A. Cahyono, J. Wu, L. Mo, M. Lv, B. Lan, Q. Meng, W. Tao, and L. Cheng, “Autonomous Ground Navigation in Highly Constrained Spaces: Lessons Learned from The 3rd BARN Challenge at ICRA 2024,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.01862>
- [29] A. Nair, F. Jiang, K. Hou, Z. Xu, S. Li, X. Xiao, and P. Stone, “DynaBARN: Benchmarking Metric Ground Navigation in Dynamic Environments,” in 2022 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR). IEEE, 2022, pp. 347–352.
- [30] N. Pérez-Higueras, R. Otero, F. Caballero, and L. Merino, “HuNavSim: A ROS 2 Human Navigation Simulator for Benchmarking Human-Aware Robot Navigation,” 2023, accepted for publication in *IEEE Robotics and Automation Letters (RA-L)*. [Online]. Available: <https://arxiv.org/abs/2305.01303>